

08. Spring Boot với Thymeleaf

8.1. Giới thiệu về Thymeleaf

Thymeleaf là một template engine hiện đại dành cho Java, được thiết kế đặc biệt để tạo ra các ứng dụng web. Nó cho phép bạn tạo ra các trang HTML động bằng cách kết hợp dữ liệu từ server với các template HTML. Điều đặc biệt của Thymeleaf là các template của nó là các file HTML hợp lệ, có thể được mở và xem trực tiếp trong trình duyệt mà không cần server, giúp việc thiết kế và prototype trở nên dễ dàng hơn.

Thymeleaf tích hợp rất tốt với Spring Boot và Spring MVC, cung cấp các tính năng mạnh mẽ như:

- **Natural Templates:** Template có thể được xem trực tiếp trong trình duyệt
- **Expression Language mạnh mẽ:** Hỗ trợ Spring EL và OGNL
- **Internationalization (i18n):** Hỗ trợ đa ngôn ngữ
- **Form Binding:** Tích hợp tốt với Spring Form
- **Fragment và Layout:** Tái sử dụng code template
- **Security Integration:** Tích hợp với Spring Security

8.2. Thêm Thymeleaf vào Spring Boot

Để sử dụng Thymeleaf trong Spring Boot, bạn chỉ cần thêm dependency `spring-boot-starter-thymeleaf` :

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Spring Boot sẽ tự động cấu hình Thymeleaf với các thiết lập mặc định:

- Template location: `src/main/resources/templates/`
- Template suffix: `.html`
- Template mode: `HTML`
- Cache: `true` trong production, `false` trong development

8.3. Cấu hình Thymeleaf

Bạn có thể tùy chỉnh cấu hình Thymeleaf trong `application.yml` :

YAML

```
spring:
  thymeleaf:
    cache: false                # Tắt cache trong development
    encoding: UTF-8
    mode: HTML
    prefix: classpath:/templates/ # Thư mục chứa template
    suffix: .html                # Phần mở rộng file template
    servlet:
      content-type: text/html
    template-resolver-order: 1
```

8.4. Cấu trúc dự án với Thymeleaf

Plain Text

```
src/
├── main/
│   ├── java/
│   │   └── com/example/demo/
│   │       ├── controller/
│   │       │   ├── HomeController.java
│   │       │   └── ProductController.java
```

```
└─ UserController.java
└─ model/
    └─ Product.java
    └─ User.java
└─ service/
    └─ ProductService.java
    └─ UserService.java
└─ DemoApplication.java
└─ resources/
    └─ static/
        └─ css/
            └─ style.css
        └─ js/
            └─ script.js
        └─ images/
    └─ templates/
        └─ fragments/
            └─ header.html
            └─ footer.html
        └─ layout/
            └─ main.html
        └─ product/
            └─ list.html
            └─ detail.html
            └─ form.html
        └─ user/
            └─ profile.html
            └─ login.html
    └─ index.html
    └─ error.html
└─ application.yml
```

8.5. Controller và Model

8.5.1. Controller cơ bản

Java

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
```

```
import java.util.List;

@Controller
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/")
    public String home(Model model) {
        model.addAttribute("message", "Welcome to our store!");
        model.addAttribute("featuredProducts",
productService.getFeaturedProducts());
        return "index"; // Trả về templates/index.html
    }

    @GetMapping("/products")
    public String listProducts(Model model,
                               @RequestParam(defaultValue = "0") int page,
                               @RequestParam(defaultValue = "10") int size) {
        List<Product> products = productService.getProducts(page, size);
        model.addAttribute("products", products);
        model.addAttribute("currentPage", page);
        model.addAttribute("totalPages", productService.getTotalPages(size));
        return "product/list";
    }

    @GetMapping("/products/{id}")
    public String productDetail(@PathVariable Long id, Model model) {
        Product product = productService.getProductById(id);
        model.addAttribute("product", product);
        model.addAttribute("relatedProducts",
productService.getRelatedProducts(id));
        return "product/detail";
    }

    @GetMapping("/products/new")
    public String newProductForm(Model model) {
        model.addAttribute("product", new Product());
        model.addAttribute("categories", productService.getAllCategories());
        return "product/form";
    }

    @PostMapping("/products")
    public String saveProduct(Product product, Model model) {
```

```

        try {
            productService.saveProduct(product);
            return "redirect:/products";
        } catch (Exception e) {
            model.addAttribute("error", "Error saving product: " +
e.getMessage());
            model.addAttribute("product", product);
            model.addAttribute("categories",
productService.getAllCategories());
            return "product/form";
        }
    }
}

```

8.5.2. Model Classes

Java

```

import java.math.BigDecimal;
import java.time.LocalDateTime;

public class Product {
    private Long id;
    private String name;
    private String description;
    private BigDecimal price;
    private String category;
    private String imageUrl;
    private boolean available;
    private LocalDateTime createdAt;

    // Constructors
    public Product() {}

    public Product(String name, String description, BigDecimal price, String
category) {
        this.name = name;
        this.description = description;
        this.price = price;
        this.category = category;
        this.available = true;
        this.createdAt = LocalDateTime.now();
    }

    // Getters and Setters
    public Long getId() { return id; }

```

```

public void setId(Long id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getDescription() { return description; }
public void setDescription(String description) { this.description =
description; }

public BigDecimal getPrice() { return price; }
public void setPrice(BigDecimal price) { this.price = price; }

public String getCategory() { return category; }
public void setCategory(String category) { this.category = category; }

public String getImageUrl() { return imageUrl; }
public void setImageUrl(String imageUrl) { this.imageUrl = imageUrl; }

public boolean isAvailable() { return available; }
public void setAvailable(boolean available) { this.available = available;
}

public LocalDateTime getCreatedAt() { return createdAt; }
public void setCreatedAt(LocalDateTime createdAt) { this.createdAt =
createdAt; }
}

```

8.6. Thymeleaf Template Syntax

8.6.1. Template cơ bản - index.html

HTML

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Welcome to Our Store</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
    <header>
        <nav>
            <a th:href="@{/}">Home</a>

```

```

        <a th:href="@{/products}">Products</a>
        <a th:href="@{/about}">About</a>
    </nav>
</header>

<main>
    <h1 th:text="${message}">Default Welcome Message</h1>

    <!-- Conditional rendering -->
    <div th:if="${featuredProducts != null and !featuredProducts.empty}">
        <h2>Featured Products</h2>
        <div class="product-grid">
            <div th:each="product : ${featuredProducts}" class="product-
card">
                
                <h3 th:text="${product.name}">Product Name</h3>
                <p
th:text="${#numbers.formatCurrency(product.price)}">$0.00</p>
                <a th:href="@{/products/{id}(id=${product.id})}"
class="btn">View Details</a>
            </div>
        </div>
    </div>

    <div th:unless="${featuredProducts != null and
!featuredProducts.empty}">
        <p>No featured products available at the moment.</p>
    </div>
</main>

<footer>
    <p>&copy; 2024 Our Store. All rights reserved.</p>
</footer>

<script th:src="@{/js/script.js}"></script>
</body>
</html>

```

8.6.2. Product List Template - product/list.html

HTML

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>

```

```

    <meta charset="UTF-8">
    <title>Products</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
    <div th:replace="fragments/header :: header"></div>

    <main class="container">
        <h1>Our Products</h1>

        <div class="actions">
            <a th:href="@{/products/new}" class="btn btn-primary">Add New
Product</a>
        </div>

        <div th:if="${products != null and !products.empty}" class="product-
list">
            <table class="table">
                <thead>
                    <tr>
                        <th>Image</th>
                        <th>Name</th>
                        <th>Category</th>
                        <th>Price</th>
                        <th>Status</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody>
                    <tr th:each="product, iterStat : ${products}"
th:class="${iterStat.odd} ? 'odd' : 'even'">
                        <td>
                            
                        </td>
                        <td th:text="${product.name}">Product Name</td>
                        <td th:text="${product.category}">Category</td>
                        <td
th:text="${#numbers.formatCurrency(product.price)}">$0.00</td>
                        <td>
                            <span th:if="${product.available}"
                                class="status available">Available</span>
                            <span th:unless="${product.available}"
                                class="status unavailable">Out of
Stock</span>
                        </td>
                    </tr>
                </tbody>
            </table>

```



```

        <a th:href="@{/products/{id}(id=${product.id})}"
            class="btn btn-sm">View</a>
        <a
th:href="@{/products/{id}/edit(id=${product.id})}"
            class="btn btn-sm btn-secondary">Edit</a>
        </td>
    </tr>
</tbody>
</table>
</div>

<div th:unless="${products != null and !products.empty}">
    <p>No products found.</p>
</div>

<!-- Pagination -->
<div th:if="${totalPages > 1}" class="pagination">
    <a th:if="${currentPage > 0}"
        th:href="@{/products(page=${currentPage - 1})}"
        class="btn">Previous</a>

    <span th:each="pageNum : ${#numbers.sequence(0, totalPages -
1)}">

        <a th:if="${pageNum != currentPage}"
            th:href="@{/products(page=${pageNum})}"
            th:text="${pageNum + 1}"
            class="btn">1</a>
        <span th:if="${pageNum == currentPage}"
            th:text="${pageNum + 1}"
            class="btn btn-current">1</span>
    </span>

    <a th:if="${currentPage < totalPages - 1}"
        th:href="@{/products(page=${currentPage + 1})}"
        class="btn">Next</a>
</div>
</main>

<div th:replace="fragments/footer :: footer"></div>
</body>
</html>

```

8.6.3. Product Form Template - `product/form.html`

HTML

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Product Form</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
    <div th:replace="fragments/header :: header"></div>

    <main class="container">
        <h1 th:text="${product.id != null} ? 'Edit Product' : 'Add New
Product'">Product Form</h1>

        <!-- Error messages -->
        <div th:if="${error}" class="alert alert-error">
            <p th:text="${error}">Error message</p>
        </div>

        <form th:action="@{/products}" th:object="${product}" method="post"
class="form">
            <input type="hidden" th:field="*{id}" />

            <div class="form-group">
                <label for="name">Product Name:</label>
                <input type="text"
                    id="name"
                    th:field="*{name}"
                    th:class="${#fields.hasErrors('name')} ? 'form-control
error' : 'form-control'"
                    required />
                <span th:if="${#fields.hasErrors('name')}"
                    th:errors="*{name}"
                    class="error-message">Name error</span>
            </div>

            <div class="form-group">
                <label for="description">Description:</label>
                <textarea id="description"
                    th:field="*{description}"
                    rows="4"
                    class="form-control"></textarea>
            </div>

            <div class="form-group">
                <label for="price">Price:</label>
                <input type="number"

```

```

        id="price"
        th:field="*{price}"
        step="0.01"
        min="0"
        class="form-control"
        required />
    <span th:if="${#fields.hasErrors('price')}}"
        th:errors="*{price}"
        class="error-message">Price error</span>
</div>

<div class="form-group">
    <label for="category">Category:</label>
    <select id="category" th:field="*{category}" class="form-
control">
        <option value="">Select Category</option>
        <option th:each="cat : ${categories}"
            th:value="${cat}"
            th:text="${cat}">Category</option>
    </select>
</div>

<div class="form-group">
    <label for="imageUrl">Image URL:</label>
    <input type="url"
        id="imageUrl"
        th:field="*{imageUrl}"
        class="form-control" />
</div>

<div class="form-group">
    <label class="checkbox-label">
        <input type="checkbox" th:field="*{available}" />
        Available for sale
    </label>
</div>

<div class="form-actions">
    <button type="submit" class="btn btn-primary">
        <span th:text="${product.id != null} ? 'Update Product' :
'Create Product'">Save</span>
    </button>
    <a th:href="@{/products}" class="btn btn-
secondary">Cancel</a>
</div>
</form>
</main>

```

```
<div th:replace="fragments/footer :: footer"></div>
</body>
</html>
```

8.7. Fragments và Layouts

8.7.1. Header Fragment - fragments/header.html

HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <!-- This head will be ignored -->
</head>
<body>
    <header th:fragment="header" class="main-header">
        <nav class="navbar">
            <div class="nav-brand">
                <a th:href="@{/}">
                    
                    <span>Our Store</span>
                </a>
            </div>

            <ul class="nav-menu">
                <li><a th:href="@{/}"
th:class="${#httpServletRequest.requestURI == '/'} ? 'active' : ''">Home</a>
</li>
                <li><a th:href="@{/products}"
th:class="${#strings.startsWith(#httpServletRequest.requestURI, '/products')}
? 'active' : ''">Products</a></li>
                <li><a th:href="@{/about}">About</a></li>
                <li><a th:href="@{/contact}">Contact</a></li>
            </ul>

            <div class="nav-actions">
                <a th:href="@{/cart}" class="cart-link">
                    <span>Cart</span>
                    <span th:if="${session.cartItemCount != null}"
th:text="${session.cartItemCount}"
class="cart-count">0</span>
                </a>
                <a th:href="@{/login}" class="login-link">Login</a>
            </div>
        </nav>
    </header>
</body>
</html>
```

```
        </div>
    </nav>
</header>
</body>
</html>
```

8.7.2. Footer Fragment - fragments/footer.html

HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<body>
    <footer th:fragment="footer" class="main-footer">
        <div class="footer-content">
            <div class="footer-section">
                <h3>Our Store</h3>
                <p>Your trusted partner for quality products.</p>
            </div>

            <div class="footer-section">
                <h3>Quick Links</h3>
                <ul>
                    <li><a th:href="@{/}">Home</a></li>
                    <li><a th:href="@{/products}">Products</a></li>
                    <li><a th:href="@{/about}">About Us</a></li>
                    <li><a th:href="@{/contact}">Contact</a></li>
                </ul>
            </div>

            <div class="footer-section">
                <h3>Contact Info</h3>
                <p>Email: info@ourstore.com</p>
                <p>Phone: (555) 123-4567</p>
            </div>
        </div>

        <div class="footer-bottom">
            <p>&copy; <span
th:text="${#dates.year(#dates.createNow())}">2024</span> Our Store. All
rights reserved.</p>
        </div>
    </footer>
</body>
</html>
```

8.8. Thymeleaf với Spring Security

8.8.1. Thêm Thymeleaf Security Extras

XML

```
<dependency>
  <groupId>org.thymeleaf.extras</groupId>
  <artifactId>thymeleaf-extras-springsecurity5</artifactId>
</dependency>
```

8.8.2. Template với Security - user/profile.html

HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
  <meta charset="UTF-8">
  <title>User Profile</title>
  <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
  <div th:replace="fragments/header :: header"></div>

  <main class="container">
    <!-- Show content only for authenticated users -->
    <div sec:authorize="isAuthenticated()">
      <h1>Welcome, <span sec:authentication="name">User</span>!</h1>

      <div class="user-info">
        <h2>Your Profile</h2>
        <p><strong>Username:</strong> <span
sec:authentication="name">username</span></p>
        <p><strong>Roles:</strong>
          <span sec:authentication="authorities">roles</span>
        </p>
      </div>

      <!-- Admin only content -->
      <div sec:authorize="hasRole('ADMIN')" class="admin-section">
        <h2>Admin Panel</h2>
        <a th:href="@{/admin/users}" class="btn">Manage Users</a>
      </div>
    </div>
  </main>
</body>
</html>
```

```

        <a th:href="@{/admin/products}" class="btn">Manage
Products</a>
    </div>

    <!-- User actions -->
    <div class="user-actions">
        <a th:href="@{/user/orders}" class="btn">My Orders</a>
        <a th:href="@{/user/settings}" class="btn">Settings</a>
        <form th:action="@{/logout}" method="post" style="display:
inline;">
            <button type="submit" class="btn btn-
danger">Logout</button>
        </form>
    </div>
</div>

<!-- Show login prompt for anonymous users -->
<div sec:authorize="isAnonymous()">
    <h1>Please Login</h1>
    <p>You need to login to access your profile.</p>
    <a th:href="@{/login}" class="btn btn-primary">Login</a>
</div>
</main>

<div th:replace="fragments/footer :: footer"></div>
</body>
</html>

```

8.9. Bản chất của Spring Boot với Thymeleaf

Bản chất của việc sử dụng Thymeleaf với Spring Boot là cung cấp một giải pháp template engine mạnh mẽ và linh hoạt để xây dựng các ứng dụng web động với giao diện người dùng phong phú. Thymeleaf không chỉ cho phép tạo ra các trang HTML động mà còn duy trì tính natural template, giúp các designer và developer có thể làm việc độc lập trên cùng một file template. Sự tích hợp chặt chẽ với Spring Boot thông qua auto-configuration giúp giảm thiểu cấu hình phức tạp, trong khi các tính năng như form binding, internationalization, và security integration làm cho Thymeleaf trở thành một lựa chọn lý tưởng cho việc phát triển các ứng dụng web full-stack với Spring Boot. Điều này đặc biệt hữu ích cho các ứng dụng cần giao diện người dùng phong phú và tương tác cao, nơi server-side rendering vẫn là một lựa chọn tối ưu.

8.8. Form Handling và Validation

Thymeleaf tích hợp rất tốt với Spring Forms, giúp việc xử lý form và hiển thị lỗi validation trở nên dễ dàng. Để sử dụng Spring Forms, bạn cần có dependency `spring-boot-starter-web` (đã bao gồm trong `spring-boot-starter-thymeleaf`).

8.8.1. Form Binding với `th:object` và `th:field`

- **`th:object`** : Liên kết form với một đối tượng trong Model. Đối tượng này sẽ được Spring populate dữ liệu từ form gửi lên.
- **`th:field`** : Liên kết một trường input trong form với một thuộc tính của đối tượng được chỉ định bởi `th:object` . Nó sẽ tự động tạo `name` , `id` và `value` cho input.

Ví dụ: `product/form.html` (đã có trong phần trước, nhưng sẽ đi sâu hơn)

HTML

```
<form th:action="@{/products}" th:object="${product}" method="post"
class="form">
    <input type="hidden" th:field="*{id}" />

    <div class="form-group">
        <label for="name">Product Name:</label>
        <input type="text"
            id="name"
            th:field="*{name}"
            th:class="${#fields.hasErrors('name')} ? 'form-control
error' : 'form-control'"
            required />
        <span th:if="${#fields.hasErrors('name')}"
            th:errors="*{name}"
            class="error-message">Name error</span>
    </div>

    <div class="form-group">
        <label for="price">Price:</label>
        <input type="number"
            id="price"
            th:field="*{price}"
            step="0.01"
            min="0"
            class="form-control">
```



```

        required />
        <span th:if="${#fields.hasErrors('\price\')}}"
            th:errors="*{price}"
            class="error-message">Price error</span>
    </div>

    <!-- Các trường khác tương tự -->

    <button type="submit" class="btn btn-primary">Save Product</button>
</form>

```

8.8.2. Validation với @Valid và BindingResult

Để thực hiện validation, bạn cần thêm dependency `spring-boot-starter-validation` :

XML

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

```

Bước 1: Thêm các Validation Annotations vào Model Class

Java

```

import javax.validation.constraints.DecimalMin;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;
import java.math.BigDecimal;

public class Product {
    private Long id;

    @NotBlank(message = "Product name is required")
    @Size(min = 3, max = 100, message = "Product name must be between 3 and
100 characters")
    private String name;

    private String description;

    @NotNull(message = "Price is required")
    @DecimalMin(value = "0.01", message = "Price must be greater than 0")
    private BigDecimal price;

```

```

@NotBlank(message = "Category is required")
private String category;

private String imageUrl;
private boolean available;

// Constructors, Getters, Setters
// ...
}

```

Bước 2: Kích hoạt Validation trong Controller

Sử dụng `@Valid` (hoặc `@Validated`) trên đối tượng model trong phương thức controller và thêm `BindingResult` ngay sau đó. `BindingResult` sẽ chứa tất cả các lỗi validation.

Java

```

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

import javax.validation.Valid;

@Controller
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping("/products/new")
    public String newProductForm(Model model) {
        model.addAttribute("product", new Product());
        model.addAttribute("categories", productService.getAllCategories());
        return "product/form";
    }

    @PostMapping("/products")
    public String saveProduct(@Valid @ModelAttribute("product") Product
product,

```

```

        BindingResult bindingResult,
        Model model) {
    if (bindingResult.hasErrors()) {
        // Nếu có lỗi validation, trả về lại form với các thông báo lỗi
        model.addAttribute("categories",
productService.getAllCategories());
        return "product/form";
    }

    try {
        productService.saveProduct(product);
        return "redirect:/products";
    } catch (Exception e) {
        // Xử lý lỗi nghiệp vụ khác nếu có
        model.addAttribute("error", "Error saving product: " +
e.getMessage());
        model.addAttribute("categories",
productService.getAllCategories());
        return "product/form";
    }
}
}

```

8.8.3. Hiện thị lỗi Validation trong Thymeleaf

Thymeleaf cung cấp các thuộc tính đặc biệt để hiện thị lỗi validation:

- **th:errors="**{fieldName}"** : Hiện thị tất cả các lỗi cho một trường cụ thể.
- **th:if="\${#fields.hasErrors('fieldName')}"** : Kiểm tra xem có lỗi cho một trường cụ thể hay không.
- **th:classappend="\${#fields.hasErrors('fieldName')} ? 'error-class' : ''"** : Thêm class CSS nếu có lỗi.
- **th:errorclass="error-class"** : Tương tự **th:classappend** nhưng chỉ áp dụng khi có lỗi.

Ví dụ: (đã có trong `product/form.html`)

HTML

```

<div class="form-group">
    <label for="name">Product Name:</label>
    <input type="text"

```

```

        id="name"
        th:field="*{name}"
        th:class="${#fields.hasErrors('name')} ? \'form-control error\'
: \'form-control\'"
        required />
<span th:if="${#fields.hasErrors('name')}"
        th:errors="*{name}"
        class="error-message">Name error</span>
</div>

```

Bạn cũng có thể hiển thị tất cả các lỗi toàn cục của form:

HTML

```

<div th:if="${#fields.hasErrors('global')}" class="alert alert-danger">
    <p th:each="err : ${#fields.globalErrors()}" th:text="${err}">Global
error</p>
</div>

```

Hoặc hiển thị tất cả các lỗi của tất cả các trường:

HTML

```

<div th:if="${#fields.hasErrors()}" class="alert alert-danger">
    <ul>
        <li th:each="err : ${#fields.allErrors()}"
th:text="${err}">Error</li>
    </ul>
</div>

```

8.8.4. Custom Validation Messages

Bạn có thể tùy chỉnh thông báo lỗi validation bằng cách tạo file `ValidationMessages.properties` (hoặc `messages.properties`) trong `src/main/resources`:

Plain Text

```

NotBlank.product.name=Tên sản phẩm không được để trống.
Size.product.name=Tên sản phẩm phải có từ {min} đến {max} ký tự.
NotNull.product.price=Giá sản phẩm không được để trống.
DecimalMin.product.price=Giá sản phẩm phải lớn hơn {value}.

```

Spring Boot sẽ tự động nhận diện và sử dụng các thông báo này.

8.9. Thymeleaf và Spring Security Integration

Thymeleaf cung cấp một module tích hợp với Spring Security, cho phép bạn hiển thị hoặc ẩn các phần tử HTML dựa trên quyền (roles) của người dùng đã đăng nhập. Điều này rất hữu ích để xây dựng các giao diện người dùng động, nơi các chức năng hoặc nội dung nhất định chỉ hiển thị cho những người dùng có quyền truy cập.

8.9.1. Thêm Dependency

Để sử dụng tích hợp này, bạn cần thêm dependency `thymeleaf-extras-springsecurity6` (hoặc phiên bản phù hợp với Spring Security của bạn) vào `pom.xml` :

XML

```
<dependency>
  <groupId>org.thymeleaf.extras</groupId>
  <artifactId>thymeleaf-extras-springsecurity6</artifactId>
  <version>3.1.1.RELEASE</version> <!-- Hoặc phiên bản mới nhất -->
</dependency>
```

8.9.2. Sử dụng trong Template

Sau khi thêm dependency, bạn cần khai báo namespace `sec` trong thẻ `<html>` của template Thymeleaf:

HTML

```
<html lang="en" xmlns:th="http://www.thymeleaf.org"
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
```

Bây giờ bạn có thể sử dụng các thuộc tính `sec:` để kiểm soát hiển thị:

Hiển thị nội dung dựa trên quyền (roles):

- `**`sec:authorize=`

8.1. Giới thiệu về Thymeleaf

Thymeleaf là một template engine phía máy chủ (server-side template engine) được phát triển cho các ứng dụng web và môi trường độc lập. Mục tiêu chính của Thymeleaf là cung cấp một cách thanh lịch và hiệu quả để tạo ra các trang web động bằng cách sử dụng các template HTML, XML, JavaScript, CSS hoặc văn bản thuần túy.

Điểm đặc biệt và nổi bật nhất của Thymeleaf là khái niệm "**Natural Templates**". Điều này có nghĩa là các template Thymeleaf là các file HTML hợp lệ và có thể được mở trực tiếp trong trình duyệt web mà không cần một máy chủ ứng dụng. Các thuộc tính và thẻ đặc biệt của Thymeleaf được thiết kế để không làm hỏng cấu trúc HTML chuẩn, cho phép các nhà thiết kế giao diện (front-end developers) làm việc trực tiếp với các file HTML mà không cần chạy ứng dụng Spring Boot. Điều này giúp tăng cường sự cộng tác giữa back-end và front-end, đồng thời đẩy nhanh quá trình phát triển.

8.1.1. Các tính năng chính của Thymeleaf

- **Natural Templating:** Như đã đề cập, template có thể được xem trực tiếp trong trình duyệt.
- **Tích hợp mạnh mẽ với Spring Framework:** Cung cấp các module tích hợp sâu với Spring MVC và Spring WebFlux, hỗ trợ Spring Expression Language (SpEL), form binding, validation, và Spring Security.
- **Expression Language:** Hỗ trợ nhiều loại biểu thức mạnh mẽ để truy cập dữ liệu, bao gồm Standard Expression Syntax (SES) và Spring Expression Language (SpEL).
- **Internationalization (i18n):** Hỗ trợ dễ dàng cho việc đa ngôn ngữ hóa ứng dụng.
- **Form Processing:** Cung cấp các tiện ích mạnh mẽ để xử lý form, bao gồm binding dữ liệu từ form đến đối tượng Java và hiển thị lỗi validation.
- **Modular Design:** Cho phép tái sử dụng các phần của template (fragments) và định nghĩa các layout chung.
- **Extensible:** Có thể mở rộng để hỗ trợ các dialect (bộ cú pháp) tùy chỉnh.

8.1.2. Tại sao chọn Thymeleaf?

- **Dễ học và sử dụng:** Cú pháp trực quan, gần gũi với HTML.
- **Thân thiện với nhà thiết kế:** Khả năng xem template trực tiếp trong trình duyệt giúp quá trình thiết kế và prototype nhanh chóng.
- **Bảo mật:** Tự động thoát (escape) các ký tự đặc biệt để ngăn chặn các cuộc tấn công XSS (Cross-Site Scripting).
- **Hiệu suất:** Cung cấp các cơ chế caching và tối ưu hóa để đảm bảo hiệu suất tốt.

8.2. Cú pháp cơ bản của Thymeleaf

Thymeleaf sử dụng các thuộc tính đặc biệt (thường bắt đầu bằng `th:`) để xử lý và hiển thị dữ liệu động. Các thuộc tính này được đặt trực tiếp vào các thẻ HTML.

8.2.1. Khai báo Namespace

Để trình duyệt và các công cụ HTML hiểu được các thuộc tính `th:`, bạn nên khai báo namespace Thymeleaf trong thẻ `<html>`:

HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
<body>
  <!-- Nội dung Thymeleaf -->
</body>
</html>
```

8.2.2. Standard Expression Syntax (SES)

Thymeleaf cung cấp một cú pháp biểu thức chuẩn để truy cập dữ liệu, URL, thông báo, v.v.

- **Biến (Variables):** `${...}`
Dùng để truy cập các biến trong context của Spring Model. Đây là cách phổ biến nhất để

hiển thị dữ liệu động.

- **Liên kết (Links/URLs):** `@{...}`

Dùng để tạo các URL động, có thể bao gồm các tham số đường dẫn (path variables) và tham số truy vấn (query parameters).

- **Thông báo (Messages):** `#{...}`

Dùng để truy cập các thông báo từ file `messages.properties` (cho Internationalization).

- **Lựa chọn (Selections):** `*{...}`

Dùng để truy cập các thuộc tính của một đối tượng đã được chọn trước đó bằng `th:object` .

- **Literal (Hằng số):**

- **Text literals:** `'Hello'` , `'Another text'` , `'Hello ' + ${user.name}`

- **Number literals:** `0` , `34` , `3.0` , `12.3`

- **Boolean literals:** `true` , `false`

- **Null literal:** `null`

8.2.3. Các thuộc tính `th:` phổ biến

- **`th:text`** : Thay thế nội dung văn bản của thẻ HTML.

- **`th:utext`** : Thay thế nội dung văn bản của thẻ HTML, nhưng không thoát các ký tự HTML đặc biệt (ví dụ: `<p>`). Cẩn thận khi sử dụng để tránh XSS.

- **`th:value`** : Đặt giá trị cho các thẻ `<input>` , `<select>` , `<textarea>` .

- **`th:href`** : Đặt giá trị cho thuộc tính `href` của thẻ `<a>` .

- **`th:src`** : Đặt giá trị cho thuộc tính `src` của thẻ `<img>` .

- **`th:if` / `th:unless`** : Hiển thị hoặc ẩn một phần tử dựa trên điều kiện.

- **`th:each`** : Lặp qua một collection hoặc array.

- **`th:object`** : Định nghĩa một đối tượng được chọn để sử dụng với cú pháp lựa chọn `*{...}` .

- **th:field** : Hỗ trợ form binding, tự động tạo `id` , `name` và `value` cho các trường form, đồng thời tích hợp với Spring Validation.
- **th:action** : Đặt giá trị cho thuộc tính `action` của thẻ `<form>` .

Việc nắm vững các cú pháp và thuộc tính cơ bản này là nền tảng để xây dựng các trang web động với Thymeleaf.

8.8. Building Dynamic Forms (Xây dựng Form động)

Thymeleaf tích hợp chặt chẽ với Spring MVC để đơn giản hóa việc xây dựng các form HTML động, cho phép bạn dễ dàng liên kết dữ liệu từ các đối tượng Java (model attributes) với các trường form và xử lý dữ liệu gửi đi.

8.8.1. `th:object` và `th:field`

Đây là hai thuộc tính quan trọng nhất khi làm việc với form trong Thymeleaf:

- **th:object="`${objectName}`"** : Được đặt trên thẻ `<form>` , nó liên kết form với một đối tượng Java cụ thể trong Spring Model. Tất cả các trường `th:field` bên trong form sẽ tham chiếu đến các thuộc tính của đối tượng này.
- **th:field="`*{propertyName}`"** : Được đặt trên các thẻ input (`<input>` , `<textarea>` , `<select>`), nó liên kết trường form với một thuộc tính của đối tượng được định nghĩa bởi `th:object` . Thymeleaf sẽ tự động:
 - Đặt thuộc tính `id` cho trường input (ví dụ: `id="propertyName"`).
 - Đặt thuộc tính `name` cho trường input (ví dụ: `name="propertyName"`).
 - Đặt thuộc tính `value` cho trường input dựa trên giá trị của `propertyName` từ đối tượng model.
 - Xử lý các trường hợp đặc biệt như checkbox, radio button.

Ví dụ:

Controller:

Java

```
@Controller
public class UserController {

    @GetMapping("/user/register")
    public String showRegistrationForm(Model model) {
        model.addAttribute("user", new User()); // Thêm đối tượng User rỗng
        // vào model
        return "user/register";
    }

    @PostMapping("/user/register")
    public String registerUser(@ModelAttribute("user") User user) {
        // Xử lý lưu user vào database
        System.out.println("Registered User: " + user.getFirstName() + " " +
            user.getLastName());
        return "redirect:/user/success";
    }
}
```

Model (User.java):

Java

```
public class User {
    private String firstName;
    private String lastName;
    private String email;
    private String password;

    // Getters and Setters
    // ...
}
```

Template (user/register.html):

HTML

```
<form action="#" th:action="@{/user/register}" th:object="${user}"
method="post">
    <label for="firstName">First Name:</label>
    <input type="text" id="firstName" th:field="*{firstName}" /><br/>

    <label for="lastName">Last Name:</label>
```

```
<input type="text" id="lastName" th:field="*{lastName}" /><br/>

<label for="email">Email:</label>
<input type="email" id="email" th:field="*{email}" /><br/>

<label for="password">Password:</label>
<input type="password" id="password" th:field="*{password}" /><br/>

<button type="submit">Register</button>
</form>
```

8.8.2. Xử lý Checkbox và Radio Buttons

`th:field` cũng đơn giản hóa việc xử lý checkbox và radio buttons:

- **Checkbox (Boolean):**
- **Checkbox (Multiple values):**
- **Radio Buttons:**

8.8.3. Dropdown Lists (`<select>`)

Thymeleaf cung cấp cách tiện lợi để tạo dropdown list từ một collection các đối tượng:

HTML

```
<label for="country">Country:</label>
<select id="country" th:field="*{country}">
  <option value="">-- Select Country --</option>
  <option th:each="country : ${allCountries}"
    th:value="${country.code}"
    th:text="${country.name}">
    Country Name
  </option>
</select>
```

Trong ví dụ này:

- `allCountries` là một `List` hoặc `Set` các đối tượng `Country` trong model.
- `th:each="country : ${allCountries}"` lặp qua từng đối tượng `country`.

- `th:value="${country.code}"` đặt giá trị cho option.
- `th:text="${country.name}"` đặt văn bản hiển thị cho option.

8.8.4. Xử lý dữ liệu Form trong Controller

Khi form được gửi (submit), Spring MVC sẽ tự động bind dữ liệu từ request parameters vào đối tượng model attribute được đánh dấu bằng `@ModelAttribute` .

Java

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ProductController {

    @GetMapping("/product/add")
    public String showAddProductForm(Model model) {
        model.addAttribute("product", new Product());
        return "product/add";
    }

    @PostMapping("/product/add")
    public String addProduct(@ModelAttribute("product") Product product) {
        // product object đã chứa dữ liệu từ form
        // Lưu product vào database
        System.out.println("New Product: " + product.getName());
        return "redirect:/products";
    }
}
```

Việc sử dụng `th:object` và `th:field` giúp giảm đáng kể lượng code boilerplate khi làm việc với form trong Spring Boot và Thymeleaf, đồng thời đảm bảo tính nhất quán và dễ bảo trì.

8.9. Validation Integration (Tích hợp Validation)

Thymeleaf tích hợp rất tốt với Spring Validation (dựa trên Bean Validation API), cho phép bạn hiển thị các thông báo lỗi validation một cách dễ dàng và thân thiện với người dùng trên

các form HTML.

8.9.1. Bean Validation API và Spring

Để sử dụng validation, bạn cần thêm dependency của Bean Validation API (ví dụ: Hibernate Validator) vào `pom.xml` :

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Sau đó, bạn có thể thêm các annotation validation vào các trường của đối tượng model (DTO/Entity) mà bạn muốn validate:

Ví dụ: `Product.java` với Validation Annotations

Java

```
import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import java.math.BigDecimal;

public class Product {
    private Long id;

    @NotBlank(message = "Tên sản phẩm không được để trống")
    @Size(min = 3, max = 100, message = "Tên sản phẩm phải có từ {min} đến {max} ký tự")
    private String name;

    @Size(max = 500, message = "Mô tả không được vượt quá {max} ký tự")
    private String description;

    @NotNull(message = "Giá sản phẩm không được để trống")
    @DecimalMin(value = "0.01", message = "Giá sản phẩm phải lớn hơn 0")
    private BigDecimal price;

    @NotBlank(message = "Danh mục không được để trống")
```

```

private String category;

private String imageUrl;
private boolean available;

// Constructors, getters, and setters
// ...
}

```

8.9.2. Kích hoạt Validation trong Controller

Để Spring thực hiện validation khi dữ liệu form được gửi, bạn cần thêm annotation `@Valid` hoặc `@Validated` trước đối tượng model attribute trong phương thức handler của Controller:

Java

```

import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ProductController {

    // ... (dependencies)

    @GetMapping("/products/new")
    public String newProductForm(Model model) {
        model.addAttribute("product", new Product());
        return "product/form";
    }

    @PostMapping("/products")
    public String saveProduct(@Valid @ModelAttribute("product") Product
product,

                                BindingResult bindingResult,
                                Model model) {

        if (bindingResult.hasErrors()) {
            // Nếu có lỗi validation, quay lại form và hiển thị lỗi
            model.addAttribute("categories",

```

```

productService.getAllCategories()); // Đảm bảo dữ liệu cần thiết cho form
    return "product/form";
}

    try {
        productService.saveProduct(product);
        return "redirect:/products";
    } catch (Exception e) {
        model.addAttribute("error", "Lỗi khi lưu sản phẩm: " +
e.getMessage());
        model.addAttribute("product", product);
        model.addAttribute("categories",
productService.getAllCategories());
        return "product/form";
    }
}
}

```

- **@Valid** : Kích hoạt quá trình validation cho đối tượng `product` .
- **BindingResult** : Đối tượng này phải luôn đi ngay sau đối tượng được validate (`@Valid Product product`). Nó chứa tất cả các lỗi validation được tìm thấy. Nếu `bindingResult.hasErrors()` trả về `true` , có nghĩa là có lỗi validation.

8.9.3. Hiện thị lỗi Validation trong Thymeleaf

Thymeleaf cung cấp các thuộc tính đặc biệt để hiện thị lỗi validation:

- **th:errors="*{fieldName}"** : Hiện thị tất cả các thông báo lỗi cho một trường cụ thể. Nếu có lỗi, nó sẽ hiện thị văn bản lỗi. Nếu không có lỗi, nó sẽ không hiện thị gì.
- **th:errorclass="fieldName"** : Thêm một lớp CSS vào thẻ HTML nếu trường đó có lỗi validation. Điều này hữu ích để thay đổi kiểu dáng (ví dụ: thêm viền đỏ) cho các trường có lỗi.

Ví dụ: `product/form.html` với hiện thị lỗi Validation

HTML

```

<form th:action="@{/products}" th:object="${product}" method="post"
class="form">
    <input type="hidden" th:field="*{id}" />

```

```

<div class="form-group">
  <label for="name">Product Name:</label>
  <input type="text"
    id="name"
    th:field="*{name}"
    th:class="{#fields.hasErrors(\'name\')} ? \'form-control is-
invalid\' : \'form-control\'" />
  <div th:if="{#fields.hasErrors(\'name\')}"
    th:errors="*{name}"
    class="invalid-feedback"></div>
</div>

<div class="form-group">
  <label for="price">Price:</label>
  <input type="number"
    id="price"
    th:field="*{price}"
    step="0.01"
    min="0"
    th:class="{#fields.hasErrors(\'price\')} ? \'form-control is-
invalid\' : \'form-control\'" />
  <div th:if="{#fields.hasErrors(\'price\')}"
    th:errors="*{price}"
    class="invalid-feedback"></div>
</div>

<!-- Các trường khác tương tự -->

<div class="form-actions">
  <button type="submit" class="btn btn-primary">Save Product</button>
</div>
</form>

```

Trong ví dụ trên:

- `th:class="{#fields.hasErrors(\'name\')} ? \'form-control is-invalid\' : \'form-control\'"` : Nếu trường `name` có lỗi, lớp `is-invalid` sẽ được thêm vào input, cho phép bạn định kiểu CSS để làm nổi bật trường lỗi.
- `th:if="{#fields.hasErrors(\'name\')}"` : Chỉ hiển thị thẻ `<div>` chứa lỗi nếu có lỗi cho trường `name`.
- `th:errors="*{name}"` : Hiển thị thông báo lỗi cụ thể cho trường `name`.

8.9.4. Hiển thị tất cả lỗi (Global Errors)

Bạn cũng có thể hiển thị tất cả các lỗi không liên quan đến một trường cụ thể (global errors) hoặc tất cả các lỗi (bao gồm cả field errors):

- **Tất cả lỗi (Global và Field Errors):**
- **Chỉ lỗi Global:**

8.9.5. Tùy chỉnh thông báo lỗi

Thông báo lỗi mặc định từ các annotation validation có thể không thân thiện với người dùng. Bạn có thể tùy chỉnh chúng bằng cách sử dụng file `messages.properties` (hoặc `ValidationMessages.properties`):

src/main/resources/messages.properties :

Plain Text

```
NotBlank.product.name=Tên sản phẩm không được để trống.  
Size.product.name=Tên sản phẩm phải có từ {2} đến {1} ký tự.  
NotNull.product.price=Giá sản phẩm không được để trống.  
DecimalMin.product.price=Giá sản phẩm phải lớn hơn {0}.
```

Spring sẽ tự động tìm kiếm các thông báo lỗi trong file này dựa trên key. Key thường có định dạng `ConstraintName.objectName.fieldName` hoặc `ConstraintName.fieldName`.

Việc tích hợp validation với Thymeleaf giúp bạn xây dựng các form mạnh mẽ, dễ sử dụng và có khả năng phản hồi tốt với người dùng, cải thiện trải nghiệm tổng thể của ứng dụng web.

8.10. Reusing HTML Code with Fragments (Tái sử dụng mã HTML với Fragments)

Một trong những tính năng mạnh mẽ của Thymeleaf là khả năng tái sử dụng các phần của mã HTML thông qua "fragments" và "layouts". Điều này giúp bạn tránh lặp lại code, làm cho template dễ quản lý và bảo trì hơn.

8.10.1. Khái niệm Fragment

Fragment là một phần nhỏ của mã HTML có thể được định nghĩa trong một file template và sau đó được nhúng vào các template khác. Một fragment được định nghĩa bằng cách sử dụng thuộc tính `th:fragment` trên một thẻ HTML.

Ví dụ: `fragments/header.html`

HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <!-- Head này sẽ bị bỏ qua khi nhúng fragment -->
</head>
<body>
  <header th:fragment="myHeader" class="main-header">
    <nav>
      <a th:href="@{/}">Home</a>
      <a th:href="@{/products}">Products</a>
      <a th:href="@{/about}">About</a>
    </nav>
  </header>
</body>
</html>
```

Trong ví dụ trên, `th:fragment="myHeader"` định nghĩa một fragment có tên `myHeader` trong thẻ `<header>`. Bạn có thể đặt tên fragment theo bất kỳ quy ước nào bạn muốn.

8.10.2. Nhúng Fragment

Để nhúng một fragment vào một template khác, bạn sử dụng thuộc tính `th:insert` hoặc `th:replace`.

- **`th:insert`** : Chèn nội dung của fragment vào bên trong thẻ mà nó được đặt. Thẻ chứa `th:insert` vẫn được giữ lại.
- **`th:replace`** : Thay thế hoàn toàn thẻ mà nó được đặt bằng nội dung của fragment. Thẻ chứa `th:replace` sẽ bị loại bỏ.
- **Cú pháp** `~{...}` : Cú pháp này cho biết Thymeleaf rằng bạn đang tham chiếu đến một template fragment. `fragments/header` là đường dẫn đến file template, và `myHeader` là tên của fragment.

8.10.3. Truyền tham số vào Fragment

Bạn có thể truyền dữ liệu vào fragment giống như cách bạn truyền dữ liệu vào một template thông thường:

Ví dụ: `fragments/alert.html`

HTML

```
<div th:fragment="alert(type, message)" th:class="'alert alert-' + ${type}">
  <p th:text="${message}"></p>
</div>
```

Nhúng và truyền tham số:

HTML

```
<body>
  <div th:replace="~{fragments/alert :: alert('success', 'Operation
completed successfully!')}"></div>
  <div th:replace="~{fragments/alert :: alert('danger', ${errorMessage})}">
</div>
</body>
```

8.10.4. Layouts (Bố cục)

Layout là một dạng đặc biệt của fragment, nơi bạn định nghĩa cấu trúc chung của trang (ví dụ: header, footer, sidebar) và để lại các "lỗ hổng" (holes) để nội dung cụ thể của từng trang con có thể được chèn vào. Thymeleaf không có một khái niệm "layout" riêng biệt như một số template engine khác, nhưng bạn có thể đạt được điều này bằng cách sử dụng fragments và `th:insert` / `th:replace`.

Ví dụ: `layout/main.html` (Layout chính)

HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title th:replace="~{fragments/title :: pageTitle}">Default Title</title>
```

```

<link rel="stylesheet" th:href="@{/css/style.css}">
<!-- Các CSS/JS chung khác -->
</head>
<body>
  <header th:replace="~{fragments/header :: myHeader}"></header>

  <main>
    <div th:replace="${content}"></div> <!-- Đây là nơi nội dung cụ thể
của trang con sẽ được chèn vào -->
  </main>

  <footer th:replace="~{fragments/footer :: myFooter}"></footer>
</body>
</html>

```

Ví dụ: `fragments/title.html` (Fragment cho tiêu đề trang)

HTML

```
<title th:fragment="pageTitle">My Application</title>
```

Ví dụ: `product/list.html` (Trang con sử dụng layout)

HTML

```

<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<body>
  <div th:replace="~{layout/main :: main(content=~{::content})}">
    <div th:fragment="content">
      <h1>Product List</h1>
      <!-- Nội dung cụ thể của trang Product List -->
      <table>
        <!-- ... -->
      </table>
    </div>
    <div th:fragment="pageTitle">Product List - My App</div>
  </div>
</body>
</html>

```

Trong ví dụ trên:

- `layout/main :: main(content=~{::content})` là một cách gọi layout `main` và truyền vào nó một fragment `content` được định nghĩa ngay trong trang `product/list.html` .
- `th:fragment="pageTitle"` trong `product/list.html` sẽ được sử dụng để thay thế `Default Title` trong layout.

Sử dụng fragments và layouts là một thực hành tốt để xây dựng các ứng dụng web lớn, giúp duy trì cấu trúc code sạch sẽ, dễ đọc và dễ mở rộng.

8.1. Giới thiệu về Thymeleaf (Lý thuyết chuyên sâu)

Thymeleaf không chỉ là một template engine đơn thuần; nó là một bộ xử lý template phía máy chủ (server-side template engine) được thiết kế để hoạt động tốt trong môi trường web (Servlet API) và cũng có thể được sử dụng trong các môi trường độc lập. Điểm nổi bật của Thymeleaf là khả năng xử lý các template HTML một cách tự nhiên (Natural Templates), nghĩa là các template này vẫn là các file HTML hợp lệ và có thể được mở trực tiếp trong trình duyệt mà không cần phải chạy ứng dụng Spring Boot. Điều này mang lại lợi ích lớn cho các nhà thiết kế giao diện (UI/UX designers) vì họ có thể làm việc với các file HTML mà không cần phụ thuộc vào backend developer, giúp tăng tốc quá trình phát triển và cộng tác.

8.1.1. Triết lý

8.1.2. Triết lý "Natural Templates"

Triết lý "Natural Templates" là một trong những đặc điểm nổi bật nhất của Thymeleaf. Nó có nghĩa là các template Thymeleaf (thường là các file `.html`) vẫn là các file HTML hợp lệ theo chuẩn W3C. Điều này cho phép các nhà thiết kế giao diện (UI/UX designers) có thể mở và xem trực tiếp các file template trong trình duyệt web mà không cần một máy chủ ứng dụng (application server) hoặc một backend đang chạy. Các thuộc tính Thymeleaf sẽ bị trình duyệt bỏ qua khi hiển thị tĩnh, nhưng sẽ được Thymeleaf processor xử lý khi ứng dụng chạy.

Lợi ích:

- **Thiết kế độc lập:** Designer có thể làm việc với các file HTML/CSS/JavaScript mà không cần kiến thức về Java hoặc Spring.

- **Prototype nhanh chóng:** Dễ dàng tạo các prototype giao diện người dùng mà không cần tích hợp backend ngay lập tức.
- **Cộng tác hiệu quả:** Giảm thiểu sự phụ thuộc giữa frontend và backend team.
- **Dễ gỡ lỗi:** Có thể kiểm tra giao diện người dùng mà không cần phải chạy toàn bộ ứng dụng.

8.1.3. Tích hợp với Spring Boot

Spring Boot cung cấp khả năng tự động cấu hình mạnh mẽ cho Thymeleaf. Khi bạn thêm `spring-boot-starter-thymeleaf` vào dự án, Spring Boot sẽ tự động:

- Cấu hình `SpringTemplateEngine` và các `TemplateResolver` cần thiết.
- Thiết lập vị trí mặc định cho các template (`src/main/resources/templates/`).
- Thiết lập hậu tố file mặc định (`.html`).
- Tự động tắt cache trong môi trường phát triển (development) và bật cache trong môi trường sản xuất (production) để tối ưu hiệu suất.

Điều này giúp bạn bắt đầu phát triển ứng dụng web với Thymeleaf và Spring Boot một cách nhanh chóng mà không cần cấu hình phức tạp.

8.2. Cú pháp cơ bản của Thymeleaf (Lý thuyết chuyên sâu)

Thymeleaf sử dụng các thuộc tính đặc biệt (thường bắt đầu bằng `th:`) để thêm logic động vào các thẻ HTML tĩnh. Các thuộc tính này được gọi là **Standard Dialect** của Thymeleaf. Khi Thymeleaf xử lý template, nó sẽ thay thế hoặc sửa đổi các thuộc tính HTML gốc dựa trên giá trị của các thuộc tính `th:` .

8.2.1. Namespace Declaration

Để trình duyệt và các công cụ phát triển hiểu rằng bạn đang sử dụng các thuộc tính Thymeleaf, bạn nên khai báo namespace `xmlns:th` trong thẻ `<html>` :

HTML

```
<html lang="en" xmlns:th="http://www.thymeleaf.org">
```

Việc khai báo này không bắt buộc để Thymeleaf hoạt động, nhưng nó giúp các IDE (như IntelliJ IDEA, Eclipse) cung cấp tính năng tự động hoàn thành (autocompletion) và kiểm tra cú pháp, đồng thời giúp trình duyệt bỏ qua các thuộc tính `th:` một cách sạch sẽ.

8.2.2. Biểu thức chuẩn (Standard Expressions)

Thymeleaf cung cấp một số loại biểu thức để truy cập dữ liệu và thực hiện các thao tác khác nhau:

1. Biểu thức biến (Variable Expressions): `${...}`

- Dùng để truy cập các biến trong ngữ cảnh của template (thường là các thuộc tính được thêm vào `Model` trong Spring MVC).
- Có thể truy cập các thuộc tính của đối tượng, gọi phương thức, hoặc truy cập các phần tử trong collection.
- Ví dụ:**

2. Biểu thức lựa chọn (Selection Expressions): `*{...}`

- Tương tự như biểu thức biến, nhưng được sử dụng để truy cập các thuộc tính của một đối tượng đã được chọn trước đó (thường là đối tượng form).
- Thường được sử dụng với `th:object`.
- Ví dụ:**

3. Biểu thức liên kết (Link URLs): `@{...}`

- Dùng để tạo các URL động, có thể bao gồm context path, path variables và request parameters.
- Thymeleaf sẽ tự động mã hóa URL và thêm context path nếu cần.
- Ví dụ:**

4. Biểu thức thông điệp (Message Expressions): `#{...}`

- Dùng để truy cập các thông điệp từ file `messages.properties` (cho Internationalization - i18n).
- **Ví dụ:**

5. Biểu thức đoạn (Fragment Expressions): `~{...}`

- Dùng để nhúng các đoạn HTML (fragments) từ các file template khác.
- **Ví dụ:**

8.2.3. Các thuộc tính `th:` phổ biến

- `th:text` : Thay thế nội dung văn bản của thẻ bằng giá trị của biểu thức.
- `th:utext` : Tương tự `th:text` nhưng không thoát các ký tự HTML đặc biệt (ví dụ: `<p>`).
Cần thận khi sử dụng để tránh lỗ hổng XSS.
- `th:value` : Đặt giá trị cho các thẻ `<input>` , `<select>` , `<textarea>` .
- `th:action` : Đặt URL cho thuộc tính `action` của thẻ `<form>` .
- `th:href` : Đặt URL cho thuộc tính `href` của thẻ `<a>` hoặc `<link>` .
- `th:src` : Đặt URL cho thuộc tính `src` của thẻ `<img>` hoặc `<script>` .
- `th:if` / `th:unless` : Hiển thị hoặc ẩn một phần tử dựa trên điều kiện.
- `th:each` : Lặp qua một collection hoặc mảng.
- `th:object` : Chọn một đối tượng làm đối tượng mặc định cho các biểu thức lựa chọn `*{...}` bên trong.
- `th:field` : Tích hợp với Spring Forms, tự động tạo `id` , `name` , và `value` cho các trường input, đồng thời xử lý hiển thị lỗi.

8.2.4. Tiện ích đối tượng (Utility Objects)

Thymeleaf cung cấp một số đối tượng tiện ích (utility objects) được truy cập thông qua `#` để thực hiện các thao tác phổ biến:

- **#strings** : Thao tác với chuỗi (ví dụ: `#strings.toUpperCase(text)`).
- **#numbers** : Định dạng số (ví dụ: `#numbers.formatCurrency(price)`).
- **#dates** : Định dạng ngày tháng (ví dụ: `#dates.format(date, 'yyyy-MM-dd')`).
- **#lists** : Thao tác với danh sách (ví dụ: `#lists.size(list)`).
- **#arrays** : Thao tác với mảng.
- **#booleans** : Thao tác với boolean.
- **#objects** : Thao tác với đối tượng.
- **#uris** : Mã hóa URL.
- **#maps** : Thao tác với map.
- **#sets** : Thao tác với set.

Ví dụ:

HTML

```
<p th:text="${#dates.format(currentDate, 'dd MMMM yyyy')}"></p>
<p th:text="${#strings.capitalizeWords(product.name)}"></p>
```

Việc nắm vững các cú pháp và biểu thức cơ bản này là nền tảng để bạn có thể xây dựng các template động và tương tác với dữ liệu từ backend một cách hiệu quả trong Thymeleaf.

8.3. Tạo liên kết động (URLs) với `@{...}`

Trong các ứng dụng web, việc tạo các liên kết (URLs) một cách chính xác và linh hoạt là rất quan trọng. Thymeleaf cung cấp cú pháp `@{...}` (Link URL Expression) để tạo các URL động, giúp bạn dễ dàng quản lý các đường dẫn trong ứng dụng Spring Boot của mình. Cú pháp này không chỉ đơn thuần là nối chuỗi mà còn xử lý các vấn đề như context path, tham số đường dẫn (path variables) và tham số truy vấn (query parameters) một cách tự động và an toàn.

8.3.1. Cấu trúc cơ bản của `@{...}`

Cú pháp `@{...}` có thể được sử dụng trong bất kỳ thuộc tính HTML nào chấp nhận một URL, chẳng hạn như `href` (cho thẻ `<a>`, `<link>`), `src` (cho thẻ `<img>`, `<script>`), `action` (cho thẻ `<form>`), v.v.

Ví dụ cơ bản:

HTML

```
<a th:href="@{/products}">Xem tất cả sản phẩm</a>

<form th:action="@{/search}" method="get">
```

Trong các ví dụ trên, Thymeleaf sẽ tự động thêm context path của ứng dụng vào trước `/products`, `/images/logo.png`, và `/search`. Ví dụ, nếu ứng dụng của bạn được triển khai dưới context path `/my-app`, các URL sẽ trở thành `/my-app/products`, `/my-app/images/logo.png`, và `/my-app/search`.

8.3.2. URL với Path Variables

Khi bạn có các tham số trong đường dẫn URL (ví dụ: `/products/{id}`), bạn có thể truyền giá trị cho chúng bằng cách sử dụng cú pháp `(paramName=paramValue)`.

HTML

```
<a th:href="@{/products/{id}(id=${product.id})}">Chi tiết sản phẩm</a>
```

Trong ví dụ này, `${product.id}` sẽ được thay thế vào vị trí `{id}` trong URL. Nếu `product.id` là `123`, URL sẽ trở thành `/products/123`.

Bạn có thể có nhiều path variables:

HTML

```
<a th:href="@{/users/{userId}/orders/{orderId}(userId=${user.id},
orderId=${order.id})}">Xem đơn hàng</a>
```

8.3.3. URL với Query Parameters

Để thêm các tham số truy vấn (query parameters) vào URL, bạn cũng sử dụng cú pháp (paramName=paramValue) tương tự như path variables. Nếu có cả path variables và query parameters, query parameters sẽ được thêm vào sau path variables.

HTML

```
<a th:href="@{/products(category=${selectedCategory},
page=${currentPage})}">Lọc sản phẩm</a>
```

Nếu `selectedCategory` là `electronics` và `currentPage` là `2`, URL sẽ trở thành `/products?category=electronics&page=2`.

Kết hợp Path Variables và Query Parameters:

HTML

```
<a th:href="@{/products/{id}(id=${product.id}, lang='en', source='web')}">Chi tiết sản phẩm với tham số</a>
```

Nếu `product.id` là `456`, URL sẽ là `/products/456?lang=en&source=web`.

8.3.4. URL tương đối (Relative URLs)

Thymeleaf cũng hỗ trợ các URL tương đối. Tuy nhiên, việc sử dụng `@{...}` được khuyến khích vì nó xử lý context path một cách tự động, giúp ứng dụng của bạn linh hoạt hơn khi triển khai trên các môi trường khác nhau.

8.3.5. URL với Context Path

Trong một số trường hợp, bạn có thể cần tham chiếu đến tài nguyên tĩnh (CSS, JS, hình ảnh) mà không muốn chúng bị ảnh hưởng bởi context path của ứng dụng. Thymeleaf xử lý điều này một cách tự động khi bạn sử dụng `@{/path/to/resource}`. Dấu `/` ở đầu chỉ ra rằng đây là một đường dẫn tuyệt đối trong ngữ cảnh của ứng dụng web.

HTML

```
<link rel="stylesheet" th:href="@{/css/main.css}" />
<script th:src="@{/js/app.js}"></script>
```

8.3.6. Lợi ích của việc sử dụng `@{...}`

- **Tự động xử lý Context Path:** Không cần phải lo lắng về việc thêm context path thủ công, giúp mã nguồn dễ bảo trì hơn khi ứng dụng được triển khai ở các môi trường khác nhau.
- **Mã hóa URL an toàn:** Thymeleaf tự động mã hóa các giá trị tham số, ngăn chặn các vấn đề bảo mật như Cross-Site Scripting (XSS) và đảm bảo URL hợp lệ.
- **Dễ đọc và dễ hiểu:** Cú pháp rõ ràng giúp bạn dễ dàng nhận biết các liên kết động trong template.
- **Tích hợp tốt với Spring MVC:** Hoạt động liền mạch với các Controller và các tham số của Spring MVC.

Việc sử dụng `@{...}` là một thực hành tốt khi làm việc với Thymeleaf và Spring Boot, giúp tạo ra các ứng dụng web mạnh mẽ và dễ bảo trì.

8.4. Logic Template: Điều kiện và Vòng lặp

Thymeleaf cung cấp các thuộc tính mạnh mẽ để thêm logic điều kiện và vòng lặp vào các template HTML của bạn, cho phép bạn hiển thị nội dung động dựa trên dữ liệu từ backend. Điều này là cần thiết để tạo ra các giao diện người dùng linh hoạt và phản hồi.

8.4.1. Điều kiện (Conditional Logic): `th:if` , `th:unless` , `th:switch` , `th:case`

Các thuộc tính điều kiện cho phép bạn hiển thị hoặc ẩn các phần tử HTML dựa trên một biểu thức boolean.

1. **`th:if`** : Hiển thị phần tử nếu biểu thức là `true` .

- **Cú pháp:** `<tag th:if="${condition}">...</tag>`

- **Ví dụ:**

2. **th:unless** : Hiển thị phần tử nếu biểu thức là `false` (ngược lại của `th:if`).

- **Cú pháp:** `<tag th:unless="{condition}">...</tag>`

- **Ví dụ:**

3. **th:switch** và **th:case** : Cung cấp cấu trúc điều kiện tương tự như câu lệnh `switch` trong Java, cho phép bạn chọn một trong nhiều khối nội dung để hiển thị dựa trên giá trị của một biểu thức.

- **Cú pháp:**

- **Lưu ý:** `th:case="**"` là trường hợp mặc định (default case) và sẽ được chọn nếu không có `th:case` nào khác khớp.

8.4.2. Vòng lặp (Iteration Logic): **th:each**

Thuộc tính `th:each` cho phép bạn lặp qua các phần tử trong một collection (ví dụ: `List` , `Set`), mảng (array), hoặc `Map` và tạo ra các phần tử HTML tương ứng cho mỗi mục.

- **Cú pháp:** `<tag th:each="item : {collection}">...</tag>`

- `item` : Biến lặp, đại diện cho phần tử hiện tại trong mỗi lần lặp.

- `collection` : Biểu thức Thymeleaf trỏ đến collection hoặc mảng bạn muốn lặp qua.

- **Ví dụ lặp qua danh sách sản phẩm:**

8.4.3. Biến trạng thái lặp (Iteration Status Variable)

Khi sử dụng `th:each` , bạn có thể truy cập một biến trạng thái lặp (iteration status variable) để lấy thông tin về vòng lặp hiện tại. Biến này được khai báo bằng cách thêm `, statusVar` sau biến lặp chính.

- **Cú pháp:** `<tag th:each="item, statusVar : {collection}">...</tag>`

- **Các thuộc tính của biến trạng thái lặp:**

- `index` : Chỉ số (0-based) của phần tử hiện tại trong vòng lặp.

- `count` : Số thứ tự (1-based) của phần tử hiện tại trong vòng lặp.
- `size` : Tổng số phần tử trong collection.
- `current` : Phần tử hiện tại trong vòng lặp (giống như `item`).
- `even` : `true` nếu chỉ số là số chẵn (0, 2, 4, ...).
- `odd` : `true` nếu chỉ số là số lẻ (1, 3, 5, ...).
- `first` : `true` nếu đây là phần tử đầu tiên.
- `last` : `true` nếu đây là phần tử cuối cùng.
- **Ví dụ sử dụng biến trạng thái lặp:**

8.4.4. Lặp qua Map

Bạn có thể lặp qua một `Map` bằng cách lặp qua các `Map.Entry` của nó:

HTML

```
<ul>
  <li th:each="entry : ${myMap}">
    Key: <span th:text="${entry.key}"></span>,
    Value: <span th:text="${entry.value}"></span>
  </li>
</ul>
```

Hoặc truy cập trực tiếp key và value:

HTML

```
<ul>
  <li th:each="(key, value) : ${myMap}">
    Key: <span th:text="${key}"></span>,
    Value: <span th:text="${value}"></span>
  </li>
</ul>
```

Việc sử dụng các cấu trúc điều kiện và vòng lặp này là nền tảng để tạo ra các giao diện người dùng động và phản hồi, cho phép bạn hiển thị dữ liệu một cách linh hoạt và có tổ chức trong

các ứng dụng web Spring Boot với Thymeleaf.

8.5. Xây dựng Form động (Building Dynamic Forms)

Thymeleaf tích hợp rất tốt với Spring MVC để xử lý các form HTML, cho phép bạn dễ dàng tạo các form động để thu thập dữ liệu từ người dùng và gửi về backend. Sự tích hợp này giúp đơn giản hóa việc hiển thị dữ liệu từ model vào form, xử lý dữ liệu gửi đi, và hiển thị thông báo lỗi xác thực.

8.5.1. `th:object` và `th:field`

Đây là hai thuộc tính quan trọng nhất khi làm việc với form trong Thymeleaf và Spring MVC.

1. `th:object="${objectName}"`

- **Mô tả:** Được đặt trên thẻ `<form>` hoặc một khối HTML chứa các trường form. Nó liên kết form với một đối tượng trong Model của Spring MVC (được truyền từ Controller).
- Khi `th:object` được sử dụng, tất cả các biểu thức lựa chọn `*{...}` bên trong khối đó sẽ tham chiếu đến các thuộc tính của đối tượng được chỉ định.
- **Ví dụ:** Nếu bạn có một đối tượng `User` trong Model, bạn sẽ sử dụng `th:object="${user}"`.

2. `th:field="*{propertyName}"`

- **Mô tả:** Được đặt trên các thẻ input (`<input>` , `<textarea>` , `<select>`). Nó tự động tạo ra các thuộc tính `id` , `name` , và `value` cho trường input dựa trên thuộc tính của đối tượng được liên kết bởi `th:object`.
- `th:field` cũng có khả năng hiển thị lại giá trị đã nhập của người dùng sau khi form được gửi đi (ví dụ: khi có lỗi xác thực) và tích hợp với cơ chế hiển thị lỗi của Spring Validation.
- **Ví dụ:** `th:field="*{name}"` sẽ tạo ra `id="name"` , `name="name"` , và `value="${user.name}"`.

Ví dụ cơ bản về Form:

Controller:

Java

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class UserController {

    @GetMapping("/register")
    public String showRegistrationForm(Model model) {
        model.addAttribute("user", new User()); // Thêm một đối tượng User
        // trống vào Model
        return "register";
    }

    @PostMapping("/register")
    public String processRegistration(@ModelAttribute("user") User user) {
        // Xử lý dữ liệu user (lưu vào DB, v.v.)
        System.out.println("Registered User: " + user.getUsername() + ", " +
            user.getEmail());
        return "registration-success";
    }
}
```

register.html :

HTML

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Register</title>
</head>
<body>
    <h1>Register New User</h1>
    <form th:action="@{/register}" th:object="${user}" method="post">
        <div>
            <label for="username">Username:</label>
            <input type="text" id="username" th:field="*{username}" />
        </div>
```



```

    <div>
      <label for="email">Email:</label>
      <input type="email" id="email" th:field="*{email}" />
    </div>
    <div>
      <label for="password">Password:</label>
      <input type="password" id="password" th:field="*{password}" />
    </div>
    <button type="submit">Register</button>
  </form>
</body>
</html>

```

8.5.2. Xử lý Checkbox và Radio Buttons

`th:field` cũng hoạt động tốt với checkbox và radio buttons, tự động xử lý các giá trị boolean hoặc danh sách các giá trị được chọn.

Checkbox (Boolean):

HTML

```

<div>
  <label>
    <input type="checkbox" th:field="*{newsletterSubscription}" />
    Subscribe to Newsletter
  </label>
</div>

```

Checkbox (Multiple values):

HTML

```

<div th:each="role : ${allRoles}">
  <label>
    <input type="checkbox" th:field="*{selectedRoles}"
    th:value="${role.id}" /> <span th:text="${role.name}"></span>
  </label>
</div>

```

Radio Buttons:

HTML

```

<div>
  <label>
    <input type="radio" th:field="*{gender}" th:value="male" /> Male
  </label>
  <label>
    <input type="radio" th:field="*{gender}" th:value="female" /> Female
  </label>
</div>

```

8.5.3. Xử lý Dropdown (Select Boxes)

Thymeleaf cung cấp `th:each` và `th:value` để tạo các tùy chọn cho thẻ `<select>`.

HTML

```

<div>
  <label for="country">Country:</label>
  <select id="country" th:field="*{country}">
    <option value="">-- Select Country --</option>
    <option th:each="country : ${countries}"
      th:value="${country.code}"
      th:text="${country.name}">
      Country Name
    </option>
  </select>
</div>

```

8.5.4. Hiển thị lại dữ liệu sau khi gửi form

Một trong những lợi ích lớn của `th:field` là nó tự động điền lại các giá trị đã nhập của người dùng vào form khi form được gửi lại (ví dụ: do lỗi xác thực). Điều này giúp cải thiện trải nghiệm người dùng bằng cách không bắt họ phải nhập lại toàn bộ thông tin.

8.5.5. `th:action` và `th:method`

- **`th:action`** : Đặt URL cho thuộc tính `action` của form. Nên sử dụng `@{...}` để đảm bảo URL được xử lý đúng context path và các tham số.
- **`th:method`** : Đặt phương thức HTTP cho form (GET, POST). Thymeleaf không có thuộc tính `th:method` riêng; bạn sử dụng thuộc tính `method` HTML thông thường.

HTML

```
<form th:action="@{/submitData}" method="post" th:object="${data}">
    <!-- form fields -->
</form>
```

Việc sử dụng `th:object` và `th:field` là cách tiêu chuẩn và hiệu quả để xây dựng các form động trong ứng dụng Spring Boot với Thymeleaf, giúp bạn quản lý dữ liệu form và tích hợp với cơ chế xác thực của Spring một cách dễ dàng.

8.6. Tích hợp Validation (Validation Integration)

Spring Boot và Thymeleaf tích hợp mạnh mẽ với Bean Validation (JSR 380), cho phép bạn dễ dàng thêm các quy tắc xác thực vào các đối tượng form (model attributes) và hiển thị thông báo lỗi xác thực trực tiếp trên giao diện người dùng. Điều này giúp đảm bảo dữ liệu đầu vào hợp lệ trước khi xử lý nghiệp vụ.

8.6.1. Thêm Bean Validation Dependency

Để sử dụng Bean Validation, bạn cần thêm dependency `spring-boot-starter-validation` vào `pom.xml` :

XML

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

8.6.2. Định nghĩa Validation Rules trong Model

Bạn sử dụng các annotation từ gói `javax.validation.constraints` (hoặc `jakarta.validation.constraints` cho Spring Boot 3+) trên các trường của đối tượng model (DTO/Entity) mà bạn muốn xác thực.

Ví dụ: `User` Model với Validation Annotations

Java

```

import javax.validation.constraints.Email;
import javax.validation.constraints.Min;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Size;

public class User {

    private Long id;

    @NotBlank(message = "Username không được để trống")
    @Size(min = 3, max = 20, message = "Username phải có từ 3 đến 20 ký tự")
    private String username;

    @NotBlank(message = "Email không được để trống")
    @Email(message = "Email không hợp lệ")
    private String email;

    @NotBlank(message = "Password không được để trống")
    @Size(min = 6, message = "Password phải có ít nhất 6 ký tự")
    private String password;

    @Min(value = 18, message = "Tuổi phải lớn hơn hoặc bằng 18")
    private int age;

    // Getters and Setters
    // ...
}

```

8.6.3. Kích hoạt Validation trong Controller

Trong Controller, bạn sử dụng annotation `@Valid` (hoặc `@Validated`) trước đối tượng model attribute trong phương thức xử lý form. Spring MVC sẽ tự động kích hoạt quá trình xác thực khi form được gửi.

Bạn cũng cần thêm một đối tượng `BindingResult` ngay sau đối tượng `@Valid` để thu thập kết quả của quá trình xác thực. Nếu có lỗi, `BindingResult` sẽ chứa thông tin chi tiết về các lỗi đó.

Ví dụ: Controller xử lý Validation

Java

```

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;

import javax.validation.Valid; // Hoặc jakarta.validation.Valid cho Spring
Boot 3+

@Controller
public class UserController {

    @GetMapping("/register")
    public String showRegistrationForm(Model model) {
        model.addAttribute("user", new User());
        return "register";
    }

    @PostMapping("/register")
    public String processRegistration(@Valid @ModelAttribute("user") User
user,

                                     BindingResult bindingResult,
                                     Model model) {
        if (bindingResult.hasErrors()) {
            // Nếu có lỗi xác thực, quay lại form và hiển thị lỗi
            return "register";
        }

        // Xử lý dữ liệu user hợp lệ (lưu vào DB, v.v.)
        System.out.println("Registered User: " + user.getUsername() + ", " +
user.getEmail());
        return "registration-success";
    }
}

```

8.6.4. Hiển thị lỗi Validation trong Thymeleaf

Thymeleaf cung cấp các thuộc tính đặc biệt để hiển thị thông báo lỗi xác thực một cách dễ dàng:

1. `th:errors="*{fieldName}"` : Hiển thị tất cả các lỗi xác thực cho một trường cụ thể.

- Ví dụ:

2. `th:errorclass="fieldName"` : Thêm một class CSS vào trường input nếu trường đó có lỗi xác thực. Điều này giúp bạn dễ dàng tạo kiểu cho các trường bị lỗi.

- **Ví dụ:**

3. **#fields Utility Object:** Thymeleaf cung cấp đối tượng tiện ích `#fields` để truy cập thông tin về lỗi xác thực.

- `#fields.hasErrors('fieldName')` : Trả về `true` nếu trường có lỗi.
- `#fields.hasGlobalErrors()` : Trả về `true` nếu có lỗi toàn cục (không liên quan đến một trường cụ thể).
- `#fields.allErrors()` : Trả về danh sách tất cả các lỗi (bao gồm cả lỗi toàn cục và lỗi trường).
- `#fields.errors('fieldName')` : Trả về danh sách các lỗi cho một trường cụ thể.

Ví dụ hiển thị lỗi toàn cục và lỗi trường:

HTML

```
<form th:action="@{/register}" th:object="${user}" method="post">
  <!-- Hiển thị lỗi toàn cục (nếu có) -->
  <div th:if="${#fields.hasGlobalErrors()}" class="alert alert-danger">
    <p th:each="err : ${#fields.globalErrors()}" th:text="${err}"></p>
  </div>

  <div class="form-group">
    <label for="username">Username:</label>
    <input type="text" id="username" th:field="*{username}"
      th:class="${#fields.hasErrors('username')} ? 'form-control is-invalid' : 'form-control'" />
    <div th:if="${#fields.hasErrors('username')}" class="invalid-feedback">
      <span th:errors="*{username}"></span>
    </div>
  </div>

  <div class="form-group">
    <label for="email">Email:</label>
    <input type="email" id="email" th:field="*{email}"
      th:class="${#fields.hasErrors('email')} ? 'form-control is-invalid' : 'form-control'" />
  </div>
</form>
```

```

        <div th:if="${#fields.hasErrors('\email\')}}" class="invalid-
feedback">
            <span th:errors="*{email}"></span>
        </div>
    </div>

    <!-- Các trường khác -->

    <button type="submit" class="btn btn-primary">Register</button>
</form>

```

8.6.5. Custom Error Messages

Bạn có thể tùy chỉnh thông báo lỗi bằng cách sử dụng thuộc tính `message` trong các annotation validation (như ví dụ trên) hoặc bằng cách định nghĩa các thông báo trong file `messages.properties` :

Plain Text

```

# messages.properties
NotBlank.user.username=Tên người dùng không được để trống.
Size.user.username=Tên người dùng phải có từ {min} đến {max} ký tự.
Email.user.email=Địa chỉ email không hợp lệ.

```

Khi sử dụng file `messages.properties` , Thymeleaf sẽ tự động tìm kiếm thông báo lỗi phù hợp dựa trên tên annotation và tên trường.

Việc tích hợp validation trong Spring Boot với Thymeleaf là một quy trình liền mạch, giúp bạn xây dựng các form mạnh mẽ và thân thiện với người dùng, đồng thời đảm bảo tính toàn vẹn của dữ liệu.

8.7. Tái sử dụng mã HTML với Fragments (Reusing HTML Code with Fragments)

Trong phát triển web, việc tái sử dụng các khối mã HTML là một thực tiễn tốt để duy trì tính nhất quán, giảm trùng lặp và tăng khả năng bảo trì. Thymeleaf cung cấp một cơ chế mạnh mẽ để định nghĩa và nhúng các đoạn mã HTML có thể tái sử dụng, được gọi là **fragments**.

8.7.1. Định nghĩa Fragment

Một fragment là một phần của template HTML được đánh dấu bằng thuộc tính `th:fragment`. Bạn có thể định nghĩa fragment trong cùng một file template hoặc trong một file riêng biệt (thường là trong thư mục `src/main/resources/templates/fragments/`).

Cú pháp:

HTML

```
<div th:fragment="fragmentName">
    <!-- Nội dung của fragment -->
</div>

<!-- Hoặc với nhiều thẻ -->
<head th:fragment="common-header">
    <meta charset="UTF-8">
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
```

- `fragmentName`: Tên duy nhất của fragment. Bạn sẽ sử dụng tên này để tham chiếu khi nhúng fragment.

8.7.2. Nhúng Fragment

Thymeleaf cung cấp ba cách chính để nhúng fragment vào một template khác:

1. **`th:insert`**: Nhúng toàn bộ nội dung của fragment, bao gồm cả thẻ chứa `th:fragment`.
 - **Cú pháp:** `<div th:insert="~{templateName :: fragmentName}"></div>`
 - **Ví dụ:** Nếu `header.html` có `<header th:fragment="myHeader">...</header>`, thì `th:insert` sẽ nhúng cả thẻ `<header>`.
2. **`th:replace`**: Thay thế thẻ hiện tại bằng toàn bộ nội dung của fragment, bao gồm cả thẻ chứa `th:fragment`.
 - **Cú pháp:** `<div th:replace="~{templateName :: fragmentName}"></div>`
 - **Ví dụ:** Nếu `header.html` có `<header th:fragment="myHeader">...</header>`, và bạn dùng `<div th:replace="..."></div>`, thì thẻ `<div>` sẽ bị thay thế hoàn toàn bằng thẻ `<header>`.

và nội dung của nó.

3. **th:include** (Đã lỗi thời từ Thymeleaf 3.0, nên dùng **th:insert** hoặc **th:replace**): Chỉ nhúng nội dung bên trong của fragment, không bao gồm thẻ chứa **th:fragment**.

Sự khác biệt giữa **th:insert** và **th:replace**:

Giả sử bạn có fragment **myFragment** trong **fragments.html**:

HTML

```
<!-- fragments.html -->
<div id="myFragment" th:fragment="myFragment">
    <p>Nội dung của fragment.</p>
</div>
```

Sử dụng **th:insert**:

HTML

```
<!-- main.html -->
<body>
    <section th:insert="~{fragments :: myFragment}"></section>
</body>

<!-- Kết quả sau khi xử lý -->
<body>
    <section>
        <div id="myFragment">
            <p>Nội dung của fragment.</p>
        </div>
    </section>
</body>
```

Sử dụng **th:replace**:

HTML

```
<!-- main.html -->
<body>
    <section th:replace="~{fragments :: myFragment}"></section>
</body>
```

```
<!-- Kết quả sau khi xử lý -->
<body>
  <div id="myFragment">
    <p>Nội dung của fragment.</p>
  </div>
</body>
```

`th:replace` thường được sử dụng khi bạn muốn thay thế hoàn toàn một phần tử giữ chỗ bằng fragment, trong khi `th:insert` được dùng khi bạn muốn chèn fragment vào bên trong một phần tử hiện có.

8.7.3. Truyền tham số vào Fragment

Bạn có thể truyền dữ liệu (biến) vào fragment để làm cho chúng linh hoạt hơn. Điều này được thực hiện bằng cách định nghĩa các tham số trong `th:fragment` và truyền giá trị khi nhúng.

Định nghĩa fragment với tham số:

HTML

```
<!-- fragments/alert.html -->
<div th:fragment="alert(type, message)" th:class="'alert alert-' + ${type}">
  <p th:text="${message}"></p>
</div>
```

Nhúng fragment và truyền tham số:

HTML

```
<!-- main.html -->
<body>
  <div th:insert="~{fragments/alert :: alert('success', 'Thao tác thành công!')}"></div>
  <div th:insert="~{fragments/alert :: alert('danger', 'Đã xảy ra lỗi!')}">
</div>
</body>
```

8.7.4. Layouts (Bố cục)

Fragments là nền tảng cho việc xây dựng các bố cục (layouts) trong Thymeleaf. Một bố cục điển hình bao gồm một file HTML chính định nghĩa cấu trúc chung của trang (header, footer, sidebar, content area) và sử dụng fragments để chèn các phần động.

Ví dụ về Layout cơ bản:

HTML

```
<!-- layouts/mainLayout.html -->
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title th:text="${pageTitle}">Tiêu đề trang</title>
    <link rel="stylesheet" th:href="@{/css/common.css}">
    <th:block th:replace="~{fragments/head :: extra-head}"></th:block> <!--
Để chèn thêm CSS/JS riêng cho từng trang -->
</head>
<body>
    <header th:replace="~{fragments/header :: header}"></header>

    <main>
        <div th:replace="${content}"></div> <!-- Chèn nội dung chính của
trang -->
    </main>

    <footer th:replace="~{fragments/footer :: footer}"></footer>
    <th:block th:replace="~{fragments/body :: extra-body}"></th:block> <!--
Để chèn thêm JS cuối body -->
</body>
</html>
```

Sử dụng Layout trong một trang cụ thể:

HTML

```
<!-- products/list.html -->
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Danh sách sản phẩm</title>
    <th:block th:fragment="extra-head">
        <link rel="stylesheet" th:href="@{/css/products.css}">
    </th:block>
</head>
```

```

<body>
  <div th:replace=~{layouts/mainLayout :: layout(~{::title}, ~{::#product-
list-content}, ~{::head})}>
    <th:block id="product-list-content" th:fragment="content">
      <h1>Danh sách sản phẩm</h1>
      <!-- Nội dung cụ thể của trang danh sách sản phẩm -->
    </th:block>
  </div>
</body>
</html>

```

Trong ví dụ trên, `products/list.html` định nghĩa các fragment riêng của nó (`extra-head` , `product-list-content`) và sau đó sử dụng `th:replace` để nhúng `mainLayout.html` , truyền các fragment riêng này vào các vị trí giữ chỗ trong layout.

Fragments và layouts là công cụ không thể thiếu để xây dựng các ứng dụng web lớn, giúp quản lý mã HTML một cách hiệu quả, đảm bảo tính nhất quán về giao diện và giảm đáng kể công sức phát triển.

8.8. Tích hợp sâu với Spring (Deep Integration with Spring)

Thymeleaf được thiết kế để tích hợp một cách sâu sắc và liền mạch với Spring Framework, đặc biệt là Spring MVC và Spring Security. Sự tích hợp này không chỉ đơn thuần là việc hiển thị dữ liệu mà còn bao gồm việc tận dụng các tính năng mạnh mẽ của Spring để xây dựng các ứng dụng web phức tạp và an toàn.

8.8.1. Spring Standard Dialect

Khi bạn sử dụng `spring-boot-starter-thymeleaf` , Spring Boot sẽ tự động cấu hình `SpringTemplateEngine` với `SpringStandardDialect` . Dialect này cung cấp các thuộc tính và đối tượng tiện ích (utility objects) đặc biệt để làm việc với các tính năng của Spring:

- **th:field** : Như đã đề cập, đây là thuộc tính quan trọng nhất cho form binding, tự động tạo `id` , `name` , `value` và tích hợp với Spring Validation.
- **th:object** : Liên kết form với một đối tượng model attribute của Spring.
- **#fields** : Đối tượng tiện ích để truy cập thông tin lỗi xác thực từ `BindingResult` của Spring.

- **#mvc** : Đối tượng tiện ích để tạo URL cho các controller của Spring MVC (ít dùng hơn `@{...}` nhưng hữu ích cho các trường hợp đặc biệt).
- **#authentication** : Đối tượng tiện ích để truy cập thông tin xác thực từ Spring Security (sẽ được đề cập chi tiết hơn trong phần tích hợp Spring Security).

8.8.2. Form Binding và Validation với Spring MVC

Sự kết hợp giữa `th:object` , `th:field` , và `BindingResult` trong Controller là cốt lõi của việc xử lý form và validation trong Spring MVC với Thymeleaf. Spring tự động thực hiện việc:

1. **Data Binding:** Khi form được gửi, Spring MVC sẽ cố gắng ánh xạ dữ liệu từ request parameters vào các thuộc tính của đối tượng model attribute (được đánh dấu `@ModelAttribute`).
2. **Validation:** Nếu đối tượng model attribute được đánh dấu `@Valid` hoặc `@Validated` , Spring sẽ kích hoạt Bean Validation để kiểm tra dữ liệu. Các lỗi sẽ được lưu trữ trong đối tượng `BindingResult` .
3. **Error Handling:** Nếu `bindingResult.hasErrors()` là `true` , Controller có thể trả về lại view form, và Thymeleaf sẽ sử dụng `#fields` để hiển thị các thông báo lỗi tương ứng bên cạnh các trường input.

Điều này tạo ra một quy trình xử lý form rất hiệu quả và ít lỗi, giảm thiểu mã boilerplate.

8.8.3. Internationalization (i18n) với Spring MessageSource

Thymeleaf tích hợp hoàn hảo với cơ chế Internationalization của Spring thông qua `MessageSource` . Bạn có thể định nghĩa các thông điệp đa ngôn ngữ trong các file `messages_xx.properties` và sử dụng biểu thức `#{...}` trong template.

messages.properties (hoặc messages_en.properties):

Plain Text

```
welcome.message=Welcome to our application!
product.price=Price: {0} {1}
```

`messages_vi.properties` :

Plain Text

```
welcome.message=Chào mừng đến với ứng dụng của chúng tôi!  
product.price=Giá: {0} {1}
```

Trong template Thymeleaf:

HTML

```
<h1 th:text="#{welcome.message}"></h1>  
<p th:text="#{product.price(${product.price}, \'USD\')} "></p>
```

Spring sẽ tự động chọn file `messages.properties` phù hợp dựa trên locale của người dùng, và Thymeleaf sẽ hiển thị thông điệp tương ứng.

8.8.4. Tích hợp với Spring Security

Thymeleaf cung cấp một dialect riêng để tích hợp với Spring Security, cho phép bạn kiểm soát việc hiển thị nội dung dựa trên trạng thái xác thực và quyền của người dùng. Để sử dụng, bạn cần thêm dependency:

XML

```
<dependency>  
  <groupId>org.thymeleaf.extras</groupId>  
  <artifactId>thymeleaf-extras-springsecurity6</artifactId> <!-- Hoặc  
springsecurity5 tùy phiên bản Spring Security -->  
</dependency>
```

Sau đó, khai báo namespace trong template:

HTML

```
<html lang="en" xmlns:th="http://www.thymeleaf.org"  
xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
```

Các thuộc tính và đối tượng tiện ích phổ biến:

- **sec:authorize** : Hiển thị hoặc ẩn nội dung dựa trên quyền của người dùng.
- **sec:authentication** : Truy cập thông tin xác thực của người dùng.
- **#authentication** : Đối tượng tiện ích tương tự **sec:authentication** nhưng dùng trong biểu thức SpEL.

8.8.5. Custom Dialects và Processors

Nếu các dialect chuẩn của Thymeleaf và Spring không đáp ứng đủ nhu cầu, bạn có thể tạo các custom dialect và processor của riêng mình để mở rộng chức năng của Thymeleaf. Điều này cho phép bạn định nghĩa các thuộc tính **th:** hoặc các thẻ mới để xử lý logic tùy chỉnh.

Sự tích hợp sâu sắc này biến Thymeleaf thành một công cụ mạnh mẽ và linh hoạt cho việc phát triển các ứng dụng web Spring Boot, cho phép bạn xây dựng giao diện người dùng phong phú và tương tác một cách hiệu quả và an toàn.